

Multi-Fusion: Camera and LiDAR Sensor Fusion for 2D & 3D object Detection

IMGS – 789 : ML for Difficult Data

Shiv Vignesh Murthy

Project Goals

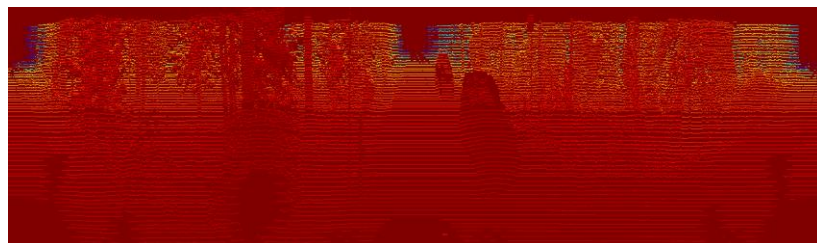
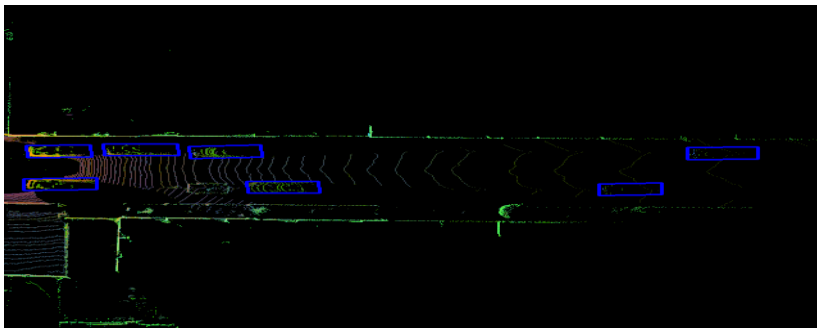
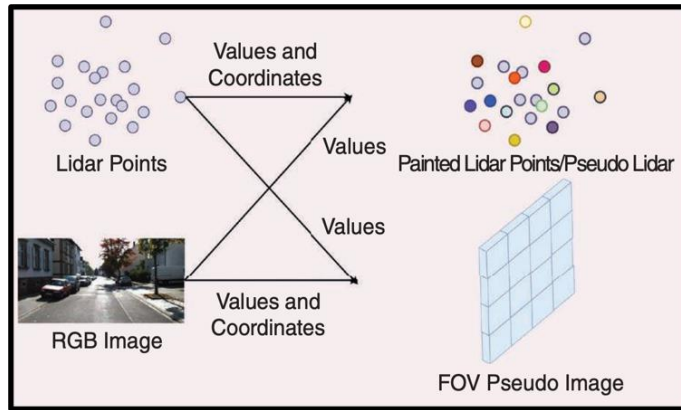
- **Designed and Developed 2D and 3D object Detection models for Autonomous Driving Scenarios.**
- **Model Architecture & Engineering**
 - **Feature Pyramid-Based Backbone:**
 - **YOLOv3** for multi-scale object detection using **Camera** and **LiDAR** modalities.
 - **Encoder-Based Backbone:**
 - **MobileNetV2** for lightweight and efficient detection, supporting both **Camera** and **LiDAR** inputs.
 - **Fusion Module:**
 - **Attention-Based Sensor Fusion** for integrating multi-modal features and improving detection robustness.
- **Datasets :**
 - **KITTI** – Benchmark suite for 2D/3D detection, tracking, and segmentation.
 - **nuScenes** – Comprehensive dataset including multi-camera, LiDAR, and radar data with annotated 3D bounding boxes.

Key Challenges in Raw LiDAR Data Processing:

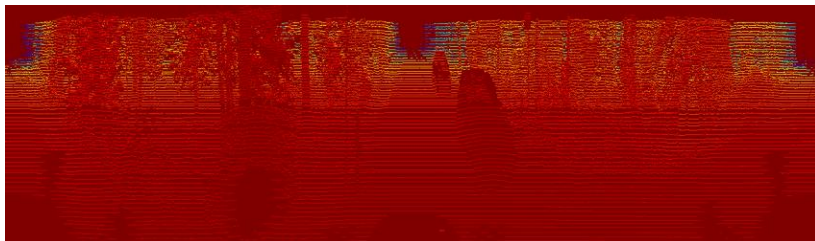
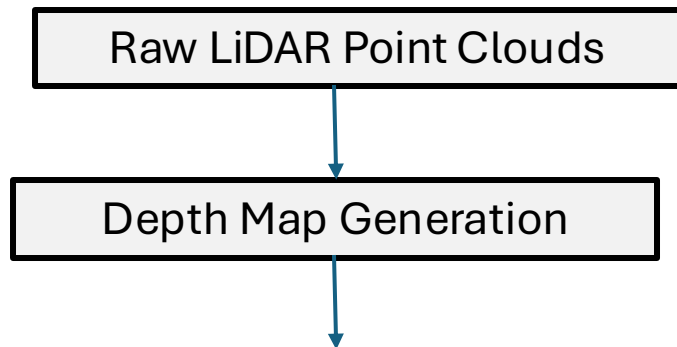
- ✓ **Unordered Nature** – LiDAR point clouds do not have a fixed structure like images, making them difficult to directly integrate with CNN-based architectures.
- ✓ **Sparsity & Irregular Density** – Points are unevenly distributed, with denser regions near the sensor and sparse areas at greater distances.
- ✓ **Lack of Semantic Information** – LiDAR captures depth and reflectance but lacks texture and color details, which are vital for accurate object recognition.

How to Process LiDAR Data?

- ✓ **Structured Representation** – Converting **LiDAR data** into **2D forms (Depth Map, Reflectance Map, or BEV representation)** makes it **compatible** with image-based models like **YOLOv3**.
- ✓ **Effective Sensor Fusion** – Aligning LiDAR features with 2D image space allows for meaningful integration with RGB image data.
- ✓ **Computational Efficiency** – Reducing raw point cloud complexity enables faster and more efficient processing within deep learning models.



Raw LiDAR Point Cloud Preprocessing



Depth Map Projection from LiDAR Point Cloud

Let each LiDAR point be represented in Cartesian coordinates as:

$$(x, y, z) \in \mathbb{R}^3$$

1. Depth Calculation (Euclidean Distance)

The depth or range from the LiDAR sensor is given by:

$$d = \sqrt{x^2 + y^2 + z^2}$$

2. Azimuthal Angle Mapping (Horizontal)

To compute the azimuthal angle (horizontal projection):

$$\theta = \arctan 2(y, x)$$

3. Elevation Angle Mapping (Vertical)

To compute the elevation angle (vertical projection):

$$\phi = \arccos\left(\frac{z}{d}\right), \quad \text{where } d = \sqrt{x^2 + y^2 + z^2}$$

4. Image Coordinates

Assuming the range of θ and ϕ is normalized to the image width W and height H , respectively:

$$x_{\text{img}} = \left\lfloor \frac{\theta - \theta_{\min}}{\theta_{\max} - \theta_{\min}} \cdot W \right\rfloor$$

$$y_{\text{img}} = \left\lfloor \frac{\phi - \phi_{\min}}{\phi_{\max} - \phi_{\min}} \cdot H \right\rfloor$$

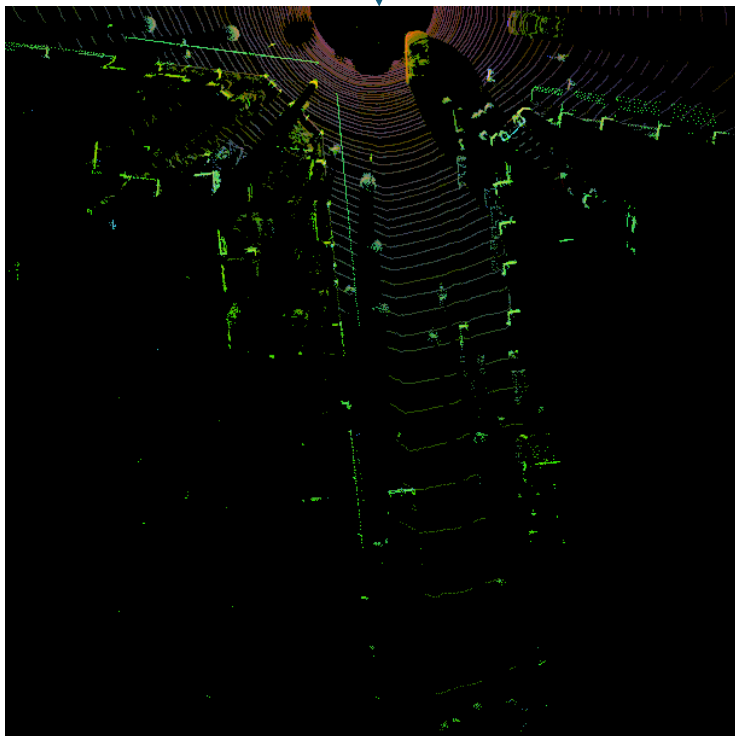
5. Depth Map Construction

The 2D depth map is created by assigning the depth value d to the corresponding image coordinate $(x_{\text{img}}, y_{\text{img}})$:

$$\text{depth_map}[y_{\text{img}}, x_{\text{img}}] = d$$

Raw LiDAR Point Clouds

Depth Map Generation



BEV Map Generation from LiDAR Point Cloud

Let a LiDAR point be denoted as (x, y, z, I) , where I is intensity. The following steps describe the Bird's-Eye View (BEV) map generation:

1. Point Cloud Filtering (ROI Clipping)

Filter points to retain only those within a predefined front-view boundary box:

$$x \in [X_{\min}, X_{\max}], \quad y \in [Y_{\min}, Y_{\max}], \quad z \in [Z_{\min}, Z_{\max}]$$

Then shift the Z coordinate:

$$z' = z - Z_{\min}$$

2. Discretization

The BEV space is discretized into a 2D grid:

$$\Delta = \frac{X_{\max} - X_{\min}}{H}$$

Each 3D point is projected onto a grid using:

$$x_{\text{bev}} = \left\lfloor \frac{x}{\Delta} \right\rfloor, \quad y_{\text{bev}} = \left\lfloor \frac{y}{\Delta} + \frac{W}{2} \right\rfloor$$

where H and W are the height and width of the BEV map.

3. Height Map

For each $(x_{\text{bev}}, y_{\text{bev}})$ cell, retain the maximum height value:

$$h(x_{\text{bev}}, y_{\text{bev}}) = \frac{z'}{Z_{\max} - Z_{\min}}$$

4. Intensity Map

The intensity map uses the reflectance value I of the top-most point:

$$I(x_{\text{bev}}, y_{\text{bev}}) = \text{intensity of top-most point}$$

5. Density Map

The number of points n falling in each cell is normalized using:

$$D(x_{\text{bev}}, y_{\text{bev}}) = \min \left(1.0, \frac{\log(n+1)}{\log(64)} \right)$$

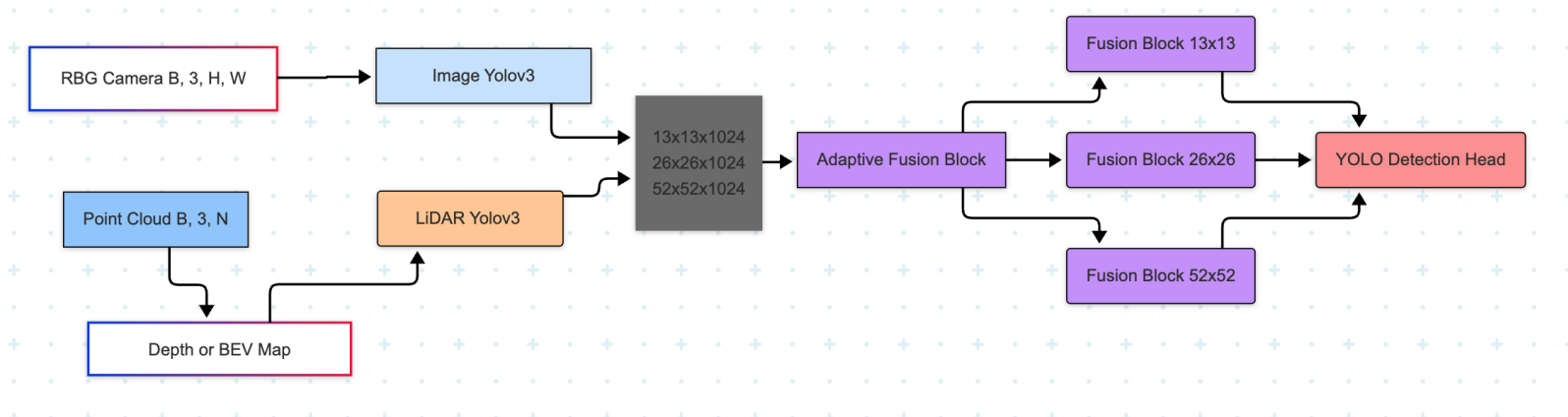
6. RGB BEV Map Construction

The three channels of the BEV map are composed as:

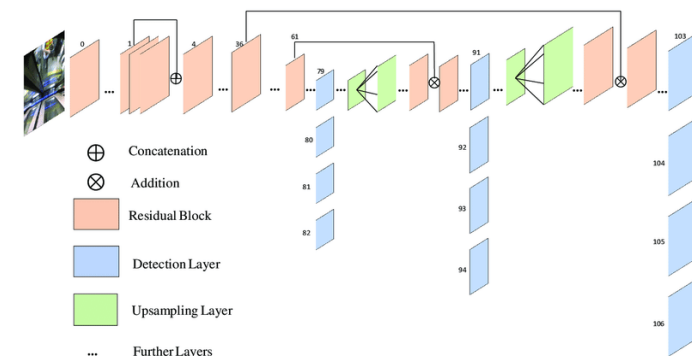
$$\text{RGB}_{\text{bev}}(x, y) = \begin{bmatrix} \text{Blue: } I(x, y) \\ \text{Green: } h(x, y) \\ \text{Red: } D(x, y) \end{bmatrix}$$

Multi-Modal Fusion Architecture

Feature Pyramid-Based Backbone



Inspired from : *Camera-LiDAR Multi-Level Sensor Fusion for Target Detection at the Network Edge*



Yolov3 Backbone Architecture

Handling LiDAR Data

Converts raw LiDAR point clouds into **structured 2D forms**:

- ✓ **Depth Map** – Encodes point distance from the sensor.
- ✓ **Bird's Eye View (BEV) Grid** – Top-down projection of LiDAR points.

Fusion Strategy

- ✓ **Parallel Backbone**: Processes RGB images and transformed LiDAR data separately.

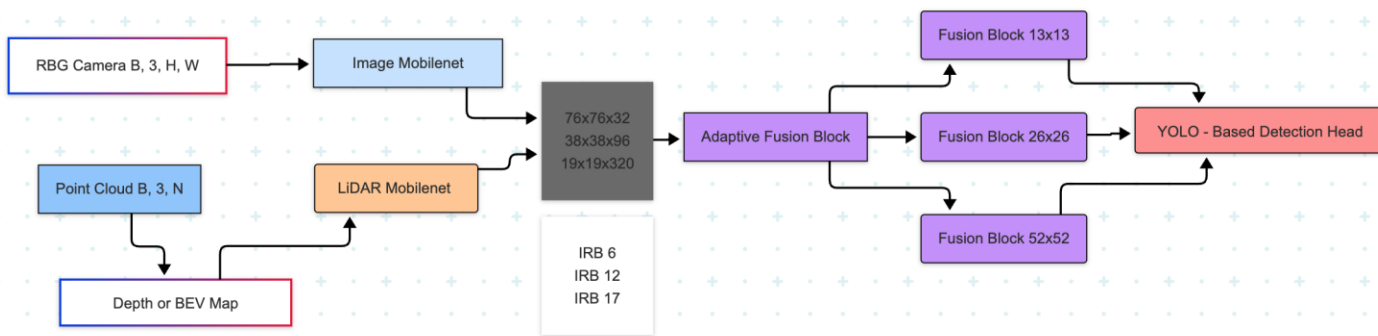
Attention Based Fusion: Combine RGB and LiDAR features before detection heads.

Highlights of Feature Pyramid Network

Feature Richness : Strong multi-scale context from deep layers fused with shallow ones.

Gradient Depth & Flow: Deep gradients flow through all pyramid levels and residual connections

Encoder-Based Backbone



Inspired from : *Camera-LiDAR Multi-Level Sensor Fusion for Target Detection at the Network Edge*

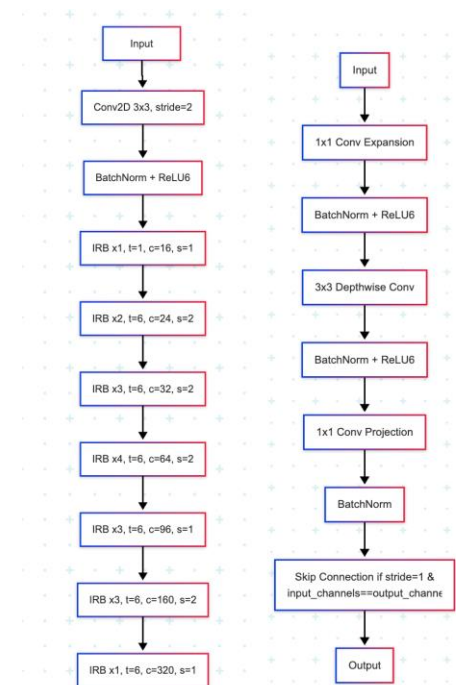
Handling LiDAR Data

Converts raw LiDAR point clouds into **structured 2D forms**:

- ✓ **Depth Map** – Encodes point distance from the sensor.
- ✓ **Bird's Eye View (BEV) Grid** – Top-down projection of LiDAR points.

Fusion Strategy

- ✓ **Parallel Backbone**: Processes RGB images and transformed LiDAR data separately.



(Left) Mobilenetv2 Backbone (Right) Single IRB Block

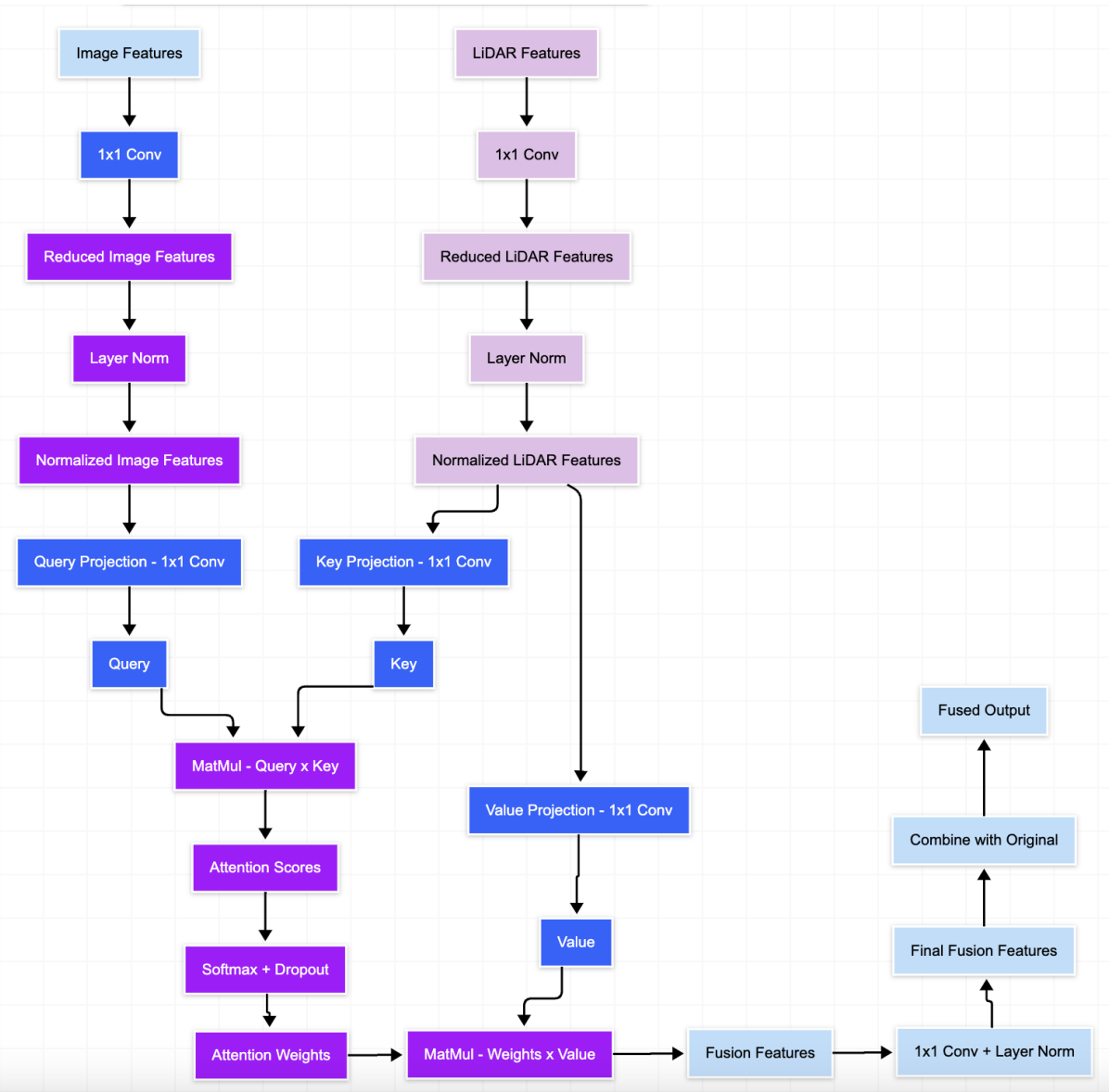
Highlights of Encoder Based Backbone

Feature Richness : Localized, efficient, but less hierarchical reuse of deep and shallow features.

Gradient Depth & Flow: Shorter gradient paths via mid-layer supervision.

Comparison

Feature	YOLOv3 (FPN)	MobileNetV2 (Encoder)
Feature Pyramid	✓ Built-in, strong for multi-scale	✗ Must be manually constructed
Speed	✗ Slower	✓ Faster
Fusion Flexibility	✗ Rigid (13/26/52)	✓ Flexible (depth-level control)
Accuracy (default)	✓ Higher for detection tasks	⚠ Lower unless tuned
Anchor Adaptability	✓ Anchors matched to feature maps	⚠ Requires tuning for custom depths



- **Dimension Reduction:** Compresses image and LiDAR features via **1×1 convolutions** for efficiency.
- **Cross-Attention:** Uses **Query, Key, and Value** to compute attention weights, highlighting important features.
- **Adaptive Fusion:** Reweights and fuses attended features dynamically.
- **Gating Mechanism (if enabled):** Learns optimal weighting of image and LiDAR features based on context.

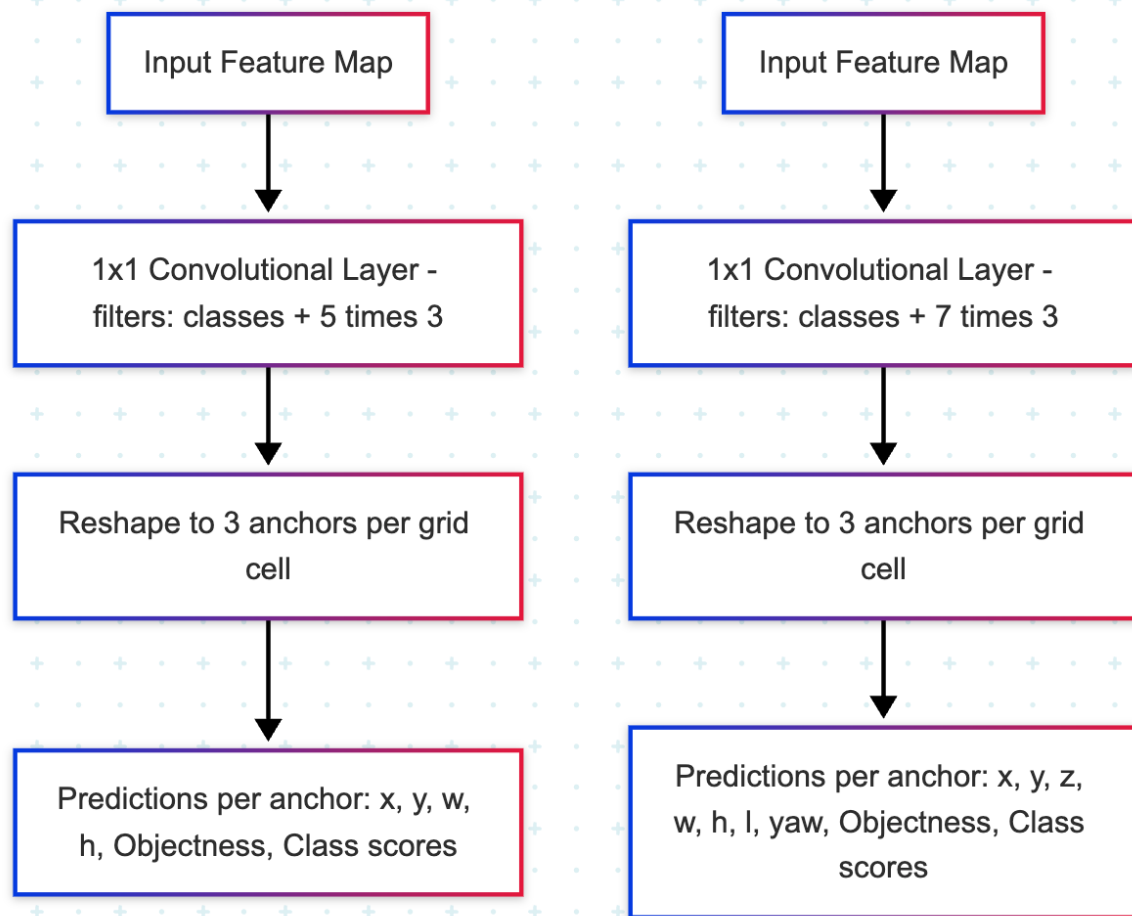
✓ **Adaptive Feature Selection** – Dynamically identifies and enhances important features.

✓ **Improved Robustness** – Suppresses irrelevant data, leading to better detection performance.

✗ **Higher Computational Cost** – Attention computations require more resources and time.

2D and 3D Loss Functions

2D and 3D Anchor Based Detection Heads



(Left) 2D Detection Head (Right) 3D Detection Head

Output Dimensions	x, y, w, h, obj, class scores	x, y, z, w, h, l, yaw, obj, class scores
Anchor Format	2D: width, height	3D: width, height, yaw
Supports Orientation	✗	✓ (Yaw angle per anchor)
Use Case	Image-based detection	LiDAR / Multi-modal 3D detection

$$\begin{aligned}
 \mathcal{L}_{\text{YOLOv3}} = & \underbrace{\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{obj}} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2]}_{\text{Localization loss (center coordinates)}} \\
 & + \underbrace{\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]}_{\text{Localization loss (width height)}} \\
 & + \underbrace{\sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2}_{\text{Objectness loss (if object exists)}} \\
 & + \underbrace{\lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{K}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2}_{\text{Objectness loss (if no object exists)}} \\
 & + \underbrace{\sum_{i=0}^{S^2} \mathbb{K}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2}_{\text{Classification loss}}
 \end{aligned}$$

- The loss function consists of 3 components
 - **Localization Loss**
Measures how well the predicted bounding box aligns with the ground truth, based on center coordinates and dimensions of the bounding box in the i -th cell and j -th anchor.
 - **Classification Loss**
Evaluates the accuracy of the predicted class probabilities against the ground truth class label for each object in the i -th cell and j -th anchor.
 - **Objectness Loss**
Determines how confidently the model predicts the presence or absence of an object in a given bounding box (i.e., for each i -th cell and j -th anchor).

Modified Localization loss

Replaced Localization loss with CloU loss using $1 - \text{IoU}$, applied directly to boxes (not separately over x, y, w, h). This is **more advanced and holistic**, capturing overlap, center distance, and aspect ratio consistency.

$$\mathcal{L}_{\text{box}} = \frac{1}{N} \sum_{i=1}^N [\mathcal{L}_x + \mathcal{L}_y + \mathcal{L}_w + \mathcal{L}_h + \mathcal{L}_{\text{eular}}]$$

$$\mathcal{L}_x = \frac{1}{N} \sum_{i=1}^N (x_i - t_{x_i})^2$$

$$\mathcal{L}_y = \frac{1}{N} \sum_{i=1}^N (y_i - t_{y_i})^2$$

$$\mathcal{L}_w = \frac{1}{N} \sum_{i=1}^N (w_i - t_{w_i})^2$$

$$\mathcal{L}_h = \frac{1}{N} \sum_{i=1}^N (h_i - t_{h_i})^2$$

$$\mathcal{L}_{\text{im}} = \frac{1}{N} \sum_{i=1}^N (\text{im}_i - t_{\text{im}_i})^2$$

$$\mathcal{L}_{\text{re}} = \frac{1}{N} \sum_{i=1}^N (\text{re}_i - t_{\text{re}_i})^2$$

$$\mathcal{L}_{\text{eular}} = \mathcal{L}_{\text{im}} + \mathcal{L}_{\text{re}}$$

$$\mathcal{L}_{\text{conf}} = \frac{1}{N} \sum_{i=1}^N \text{BCE}(\hat{p}_{\text{conf}_i}, t_{\text{conf}_i})$$

$$\mathcal{L}_{\text{cls}} = \frac{1}{N} \sum_{i=1}^N \text{BCE}(\hat{p}_{\text{cls}_i}, t_{\text{cls}_i}) \quad \text{or} \quad \text{FocalLoss}(\hat{p}_{\text{cls}_i}, t_{\text{cls}_i})$$

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{conf}} + \mathcal{L}_{\text{cls}}$$

3D Localization Loss

Measures the alignment of predicted 3D bounding box center (x,y)(x, y), dimensions (w,l)(w, l), and orientation (sinθ,cosθ)(\sin\theta, \cos\theta) with the ground truth.

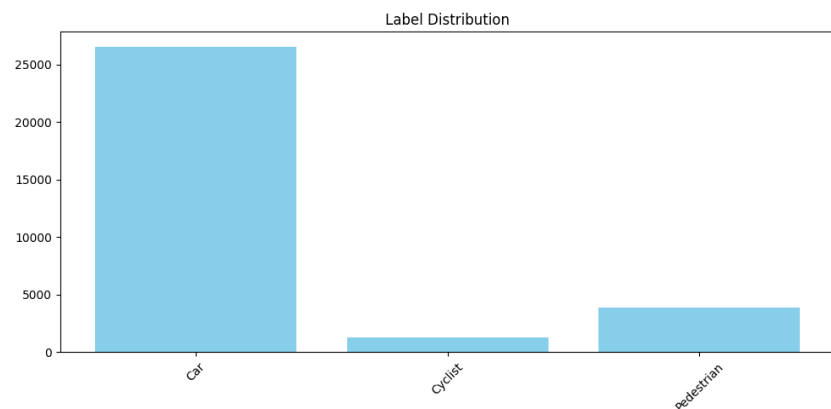
Classification Loss

Compares the predicted class probabilities with the ground truth labels using binary cross-entropy or focal loss.

Objectness Loss

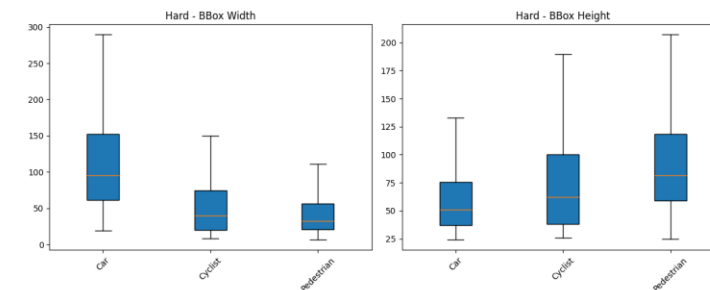
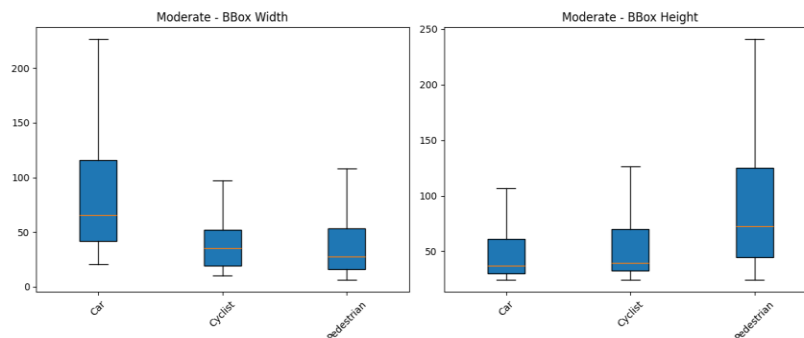
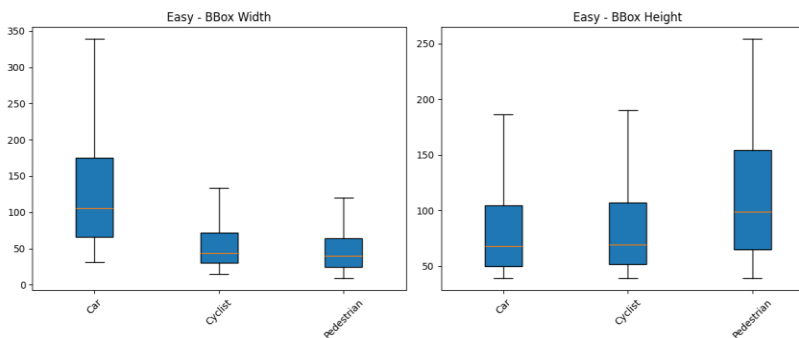
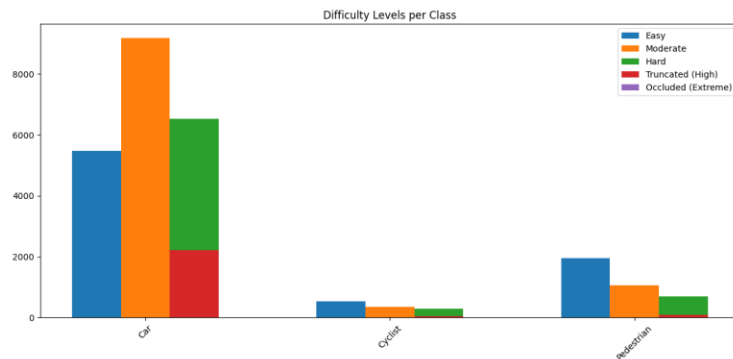
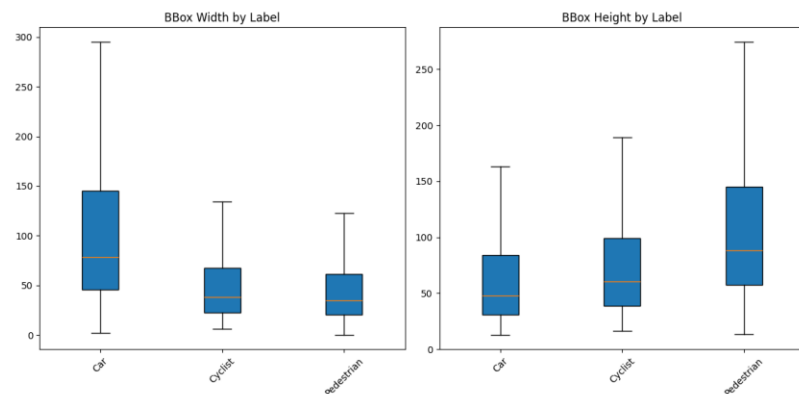
Measures the confidence of the model in predicting the presence of an object, using binary cross-entropy between predicted and target objectness values.

Datasets and Experiments

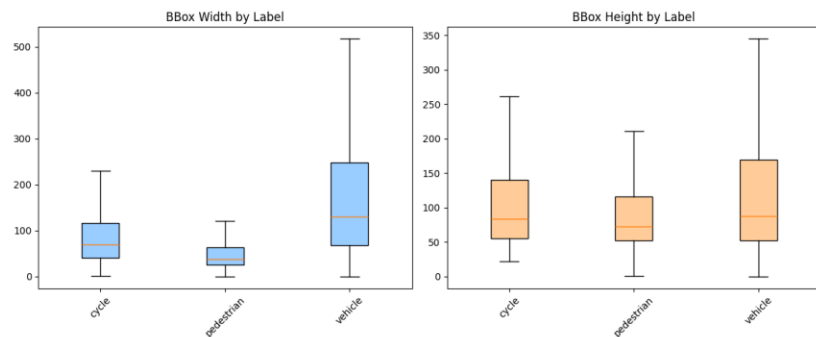
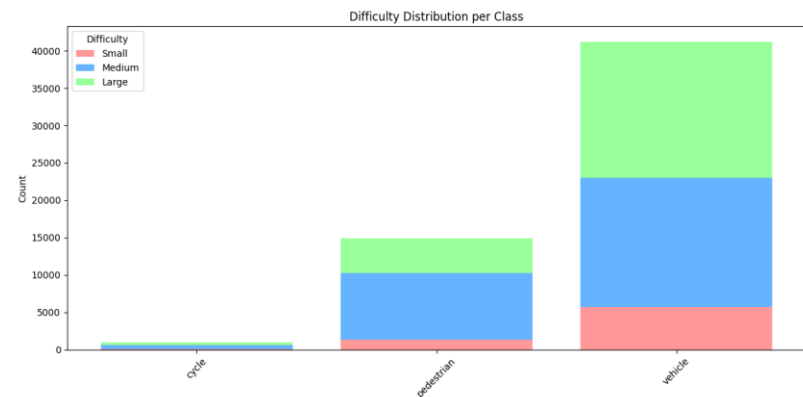
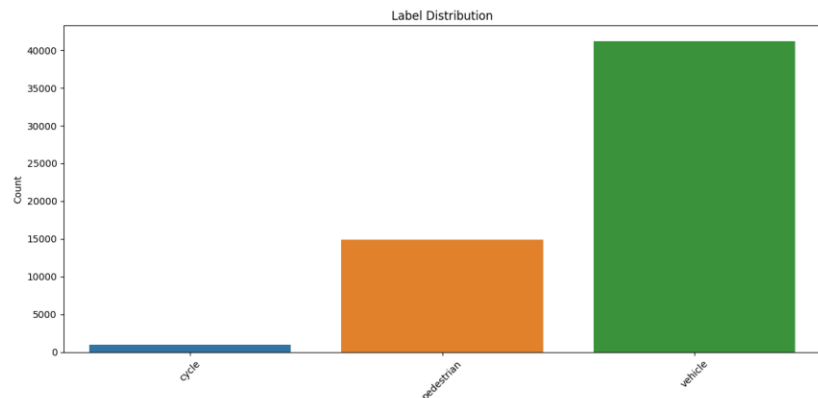


Dataset : KiTTi for 2D Object Detection

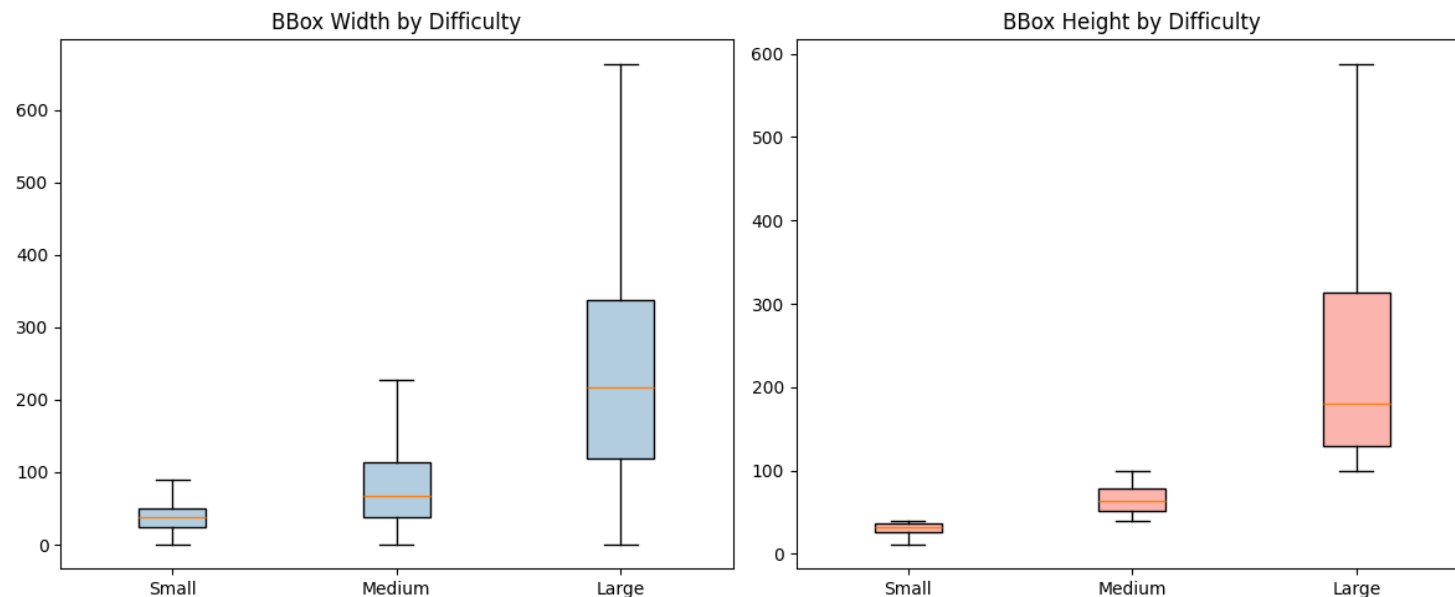
- **Benchmark Dataset:** Used for **computer vision & autonomous driving** evaluations.
- **Dataset Split:** Due to the **lack of a separate test set**, a custom split is created.
- **Challenge:** Results are **not directly comparable** to existing benchmarks.
- **Dataset Size:**
 - Training Samples: 5,985
 - Testing Samples: 1,496



NuScenes Dataset



Dataset : NuScenes Dataset for Autonomous Driving
Split – US Collected Data splits, containing 5 blobs for training and 2 blobs for validation/testing. Each blob roughly contains 3370 samples.



Comparison : kitti vs Nuscenes

Feature	KITTI	nuScenes
Publisher	Karlsruhe Institute of Technology & Toyota Technological Institute	nuTonomy (an Aptiv company)
Release Year	2012	2019
Number of Scenes	~200 sequences (~1.5 hours)	1,000 scenes (~6 hours, 20s per scene)
Sensor Setup	1 front-facing camera, 1 Velodyne HDL-64E LiDAR	6 cameras, 1 LiDAR (32-beam), 5 radars, IMU, GPS
Camera Views	Monocular/Stereo (frontal)	360° camera coverage (6 views)
LiDAR	Velodyne HDL-64E (64-beam, front)	Velodyne HDL-32E (32-beam, top-mounted)
Object Classes	8 classes (cars, pedestrians, cyclists, etc.)	23 classes (including buses, construction vehicles, traffic cones, etc.)
Annotations	2D, 3D bounding boxes, tracking	3D bounding boxes, tracking, attributes, visibility, instance-level IDs
Weather & Time Diversity	Limited (mostly clear daylight)	Diverse (day/night, rain, cloudy, etc.)
Geographic Diversity	Karlsruhe, Germany	Boston, USA & Singapore

Model Performance : KiTTi Dataset 2D Detection

(Testing Samples: 1,496)

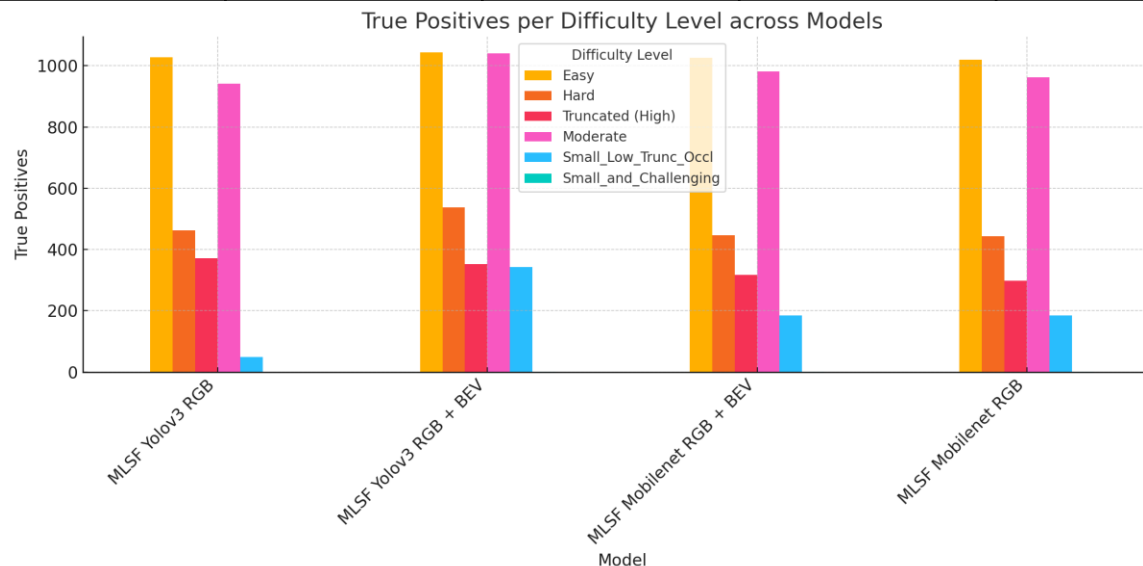
Model Variant	Class	AP	Precision	Recall	F1
YOLOv3 (RGB + BEV)	Car	0.81953	0.84265	0.83745	0.84004
	Cyclist	0.58258	0.54839	0.65385	0.59649
	Pedestrian	0.54312	0.63636	0.58333	0.60870
YOLOv3 (RGB only)	Car	0.67255	0.94033	0.68129	0.79012
	Cyclist	0.41618	0.87671	0.42809	0.57528
	Pedestrian	0.47711	0.82846	0.49708	0.62135
MobileNet (RGB only)	Car	0.66147	0.89363	0.68049	0.77263
	Cyclist	0.37150	0.68889	0.41472	0.51775
	Pedestrian	0.46209	0.64215	0.53099	0.58131
MobileNet (RGB + BEV)	Car	0.66876	0.81070	0.70411	0.75366
	Cyclist	0.34694	0.62564	0.40803	0.49393
	Pedestrian	0.43011	0.67460	0.49708	0.57239

Input Image Size – 608x608

Model Performance : KiTTi Dataset 2D Detection

(Testing Samples: 1,496)

Model	Easy	Hard	Truncated (High)	Moderate	Small_Low_Trunc_Occl	Small_and_Challenging
MLSF Yolov3 RGB	1028	463	372	942	49	0
MLSF Yolov3 RGB + BEV	1043	537	353	1041	343	1
MLSF Mobilenet RGB + BEV	1026	447	318	982	185	1
MLSF Mobilenet RGB	1020	444	298	962	184	1



Model Performance : KiTTi Dataset 3D Detection (Testing Samples: 1,496)

Model Variant	Class	AP	Precision	Recall	F1
YOLOv3 3D (BEV) - MSE Loss	Car	0.5942	0.06986	0.96249	0.13027
	Cyclist	0.3494	0.15426	0.85667	0.26144
	Pedestrian	0.3139	0.16330	0.86644	0.27480
YOLOv3 3D (BEV) - Clou Loss	Car	0.8550	0.11515	0.93963	0.20516
	Cyclist	0.1565	0.21429	0.50000	0.30000
	Pedestrian	0.0080	0.13619	0.03995	0.06178

High False Positive Rates

- A large number of predicted boxes have high confidence (>0.9), but many are incorrect.
- These overconfident predictions make it **difficult for NMS (Non-Max Suppression)** to filter out false detections effectively.

Overconfident Prediction Boxes

- The model outputs boxes with high confidence even when localization or class accuracy is poor.
- This skews the objectness loss and leads to **false positives surviving post-processing**.

Model Performance : NuScenes Dataset (Testing Samples: 6,742)

Model	Class	AP	Precision	Recall	F1
MLSF Yolov3 (RGB)	Vehicle	0.34703	0.60282	0.42249	0.49680
	Cycle	0.02082	0.33333	0.04104	0.07308
	Pedestrian	0.13688	0.38772	0.19184	0.25668
MLSF Yolov3 (RGB + BEV)	Vehicle	0.56339	0.68982	0.64655	0.66749
	Cycle	0.04442	0.41509	0.05759	0.10115
	Pedestrian	0.17935	0.43204	0.26698	0.33002
MLSF Mobilenetv2 (RGB)	Vehicle	0.15414	0.62257	0.21883	0.32384
	Cycle	0.00000	0.00000	0.00000	0.00000
	Pedestrian	0.00940	0.17169	0.03263	0.05484
MLSF Mobilenetv2 (RGB + BEV)	Vehicle	0.31297	0.66928	0.41124	0.50945
	Cycle	0.00636	0.31579	0.00785	0.01533
	Pedestrian	0.10918	0.25012	0.22377	0.23621

Cross Dataset Adaptability

Model	Class	AP	Precision	Recall	F1
YOLOv3 KITTI → NuScenes	Car	0.25418	0.79079	0.28809	0.42233
	Cyclist	0.02777	0.41975	0.04450	0.08047
	Pedestrian	0.04563	0.67775	0.05842	0.10757
YOLOv3 NuScenes → KITTI	Car	0.15501	0.71641	0.20138	0.31439
	Cyclist	0.02158	0.31707	0.04348	0.07647
	Pedestrian	0.07957	0.35843	0.13918	0.20051
RGB+LiDAR Methods	Car	0.00000	0.00000	0.00000	0.00000
	Pedestrian	0.00000	0.00000	0.00000	0.00000

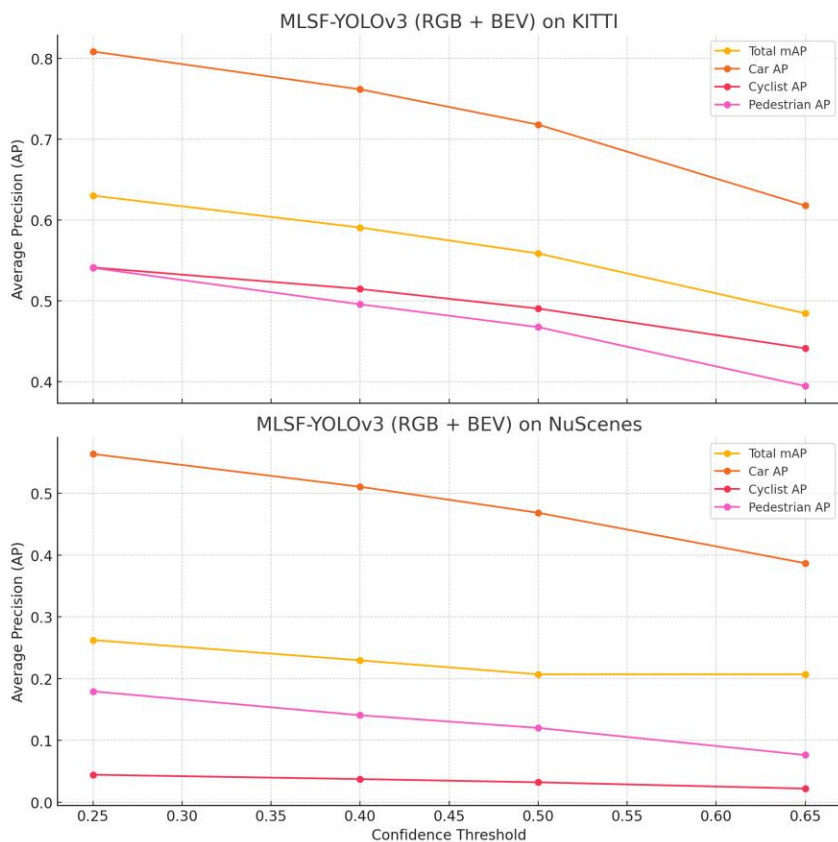
⚠ Drawback of RGB+LiDAR Methods Across Datasets

The failure of RGB+LiDAR methods in cross-dataset settings—particularly between **KITTI** and **nuScenes**—is primarily due to **significant hardware differences in LiDAR sensors**:

- **KITTI** uses a **Velodyne HDL-64E**, producing dense, 64-layer point clouds.
- **nuScenes** uses a **Velodyne VLP-32C**, offering only 32 layers and a much sparser representation.

These differences lead to a **mismatch in spatial density, range, and vertical resolution**, making models trained on one dataset poorly generalize to the other—especially when LiDAR features are tightly coupled to detection.

Confidence Thresholding Experiment



Confidence	mAP	Car AP	Cyclist AP	Pedestrian AP
0.25	0.6301	0.8084	0.5411	0.5406
0.40	0.5906	0.7616	0.5145	0.4955
0.50	0.5585	0.7179	0.4903	0.4673
0.65	0.4843	0.6177	0.4410	0.3943

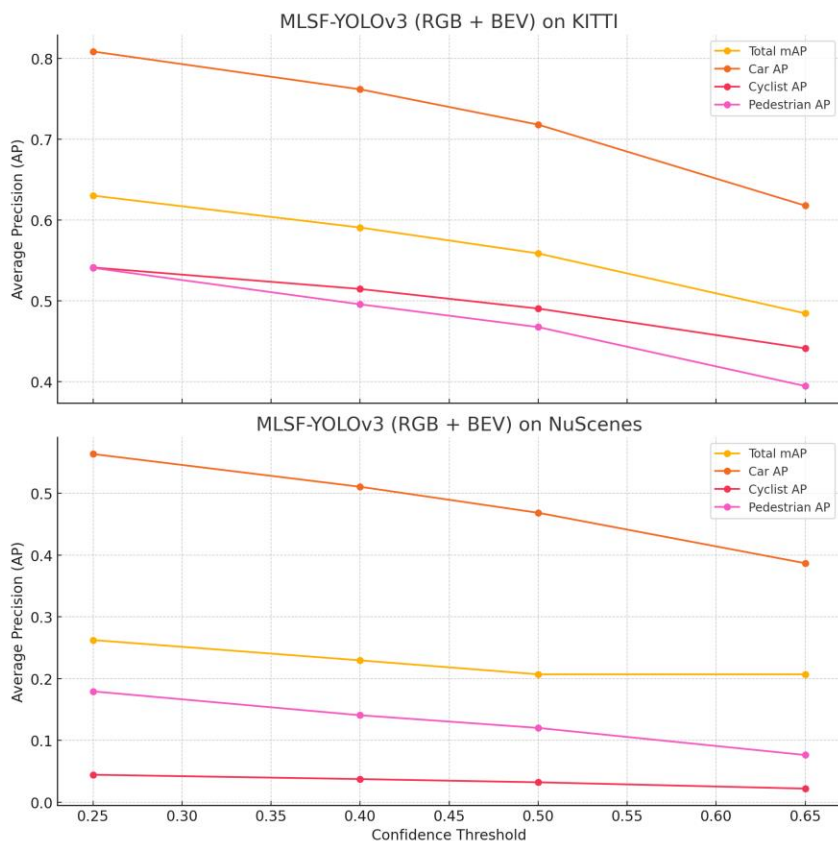
Table 1: MLSF-YOLOv3 (RGB + BEV) KITTI

Confidence	mAP	Car AP	Cyclist AP	Pedestrian AP
0.25	0.2624	0.5634	0.0444	0.1794
0.40	0.2295	0.5104	0.0374	0.1407
0.50	0.2069	0.4683	0.0322	0.1202
0.65	0.2069	0.3867	0.0220	0.0763

Table 2: MLSF-YOLOv3 (RGB + BEV) NuScenes

Confidence Thresholding Experiment – KiTTi & NuScenes

2D Detection (Testing Samples: 1,496)



Confidence	mAP	Car AP	Cyclist AP	Pedestrian AP
0.25	0.6301	0.8084	0.5411	0.5406
0.40	0.5906	0.7616	0.5145	0.4955
0.50	0.5585	0.7179	0.4903	0.4673
0.65	0.4843	0.6177	0.4410	0.3943

Table 1: MLSF-YOLOv3 (RGB + BEV) KiTTi

Confidence	mAP	Car AP	Cyclist AP	Pedestrian AP
0.25	0.2624	0.5634	0.0444	0.1794
0.40	0.2295	0.5104	0.0374	0.1407
0.50	0.2069	0.4683	0.0322	0.1202
0.65	0.2069	0.3867	0.0220	0.0763

Table 2: MLSF-YOLOv3 (RGB + BEV) NuScenes

Detection Performance — nuScenes Nighttime Split

Class	AP	Precision	Recall	F1 Score	TP	FP	FN
Car	0.06679	0.32202	0.11009	0.16408	740	1538	5982
Cyclist	0.00000	0.00000	0.00000	0.00000	0	7	352
Pedestrian	0.00022	0.02462	0.00616	0.00986	13	513	2097

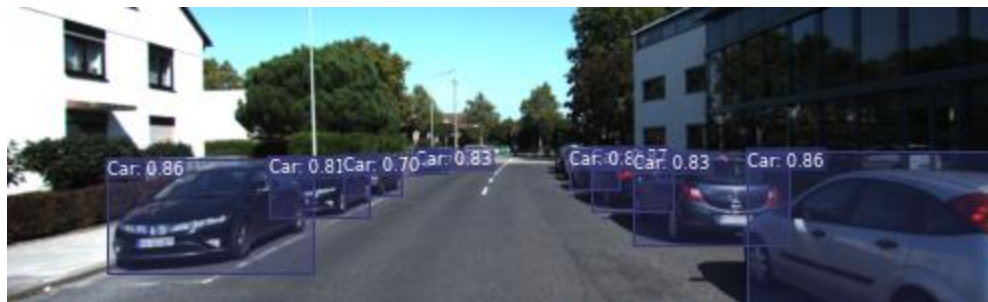
Table 1: MLSF-YOLOv3 (RGB + BEV) on nuScenes Nighttime test Split

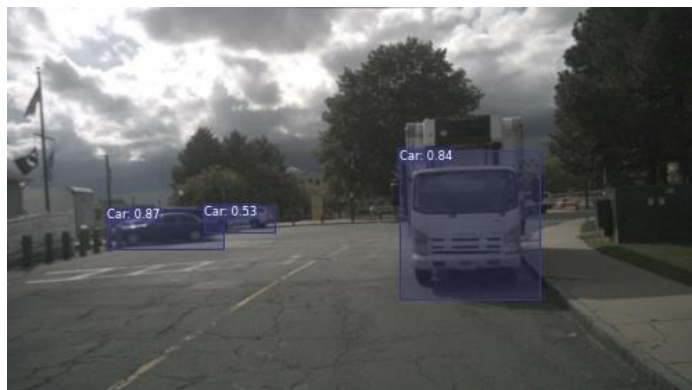
Class	AP	Precision	Recall	F1	TP	FP	FN
Car	0.01846	0.72195	0.02202	0.04273	148	57	6574
Cyclist	0.00000	0.00000	0.00000	0.00000	0	0	352
Pedestrian	0.00108	0.05220	0.00900	0.01536	19	345	2091

Table 1: MLSF-MobileNet (RGB + BEV) on nuScenes Nighttime test Split

Inference Time Comparison – KITTI Test Set (Batch Size: 12)

Model Variant	Avg Inference Time (sec)	Total Inference Time (sec)
MLSF-YOLOv3 (RGB + BEV)	0.8824	105.89
MLSF-YOLOv3 (RGB)	0.4226	50.71
MLSF-MobileNet (RGB)	0.1449	17.39
MLSF-MobileNet (RGB + BEV)	0.1699	19.39







Key Takeaways

LiDAR Improves Object Detection Across Difficulty Levels

- Fusing LiDAR with RGB significantly boosts detection, especially in *moderate*, *hard*, and *occluded* scenarios.
- As seen in the true positive breakdown, both YOLOv3 and MobileNet models detect more small and challenging objects when LiDAR is fused.
- This is because LiDAR complements RGB by providing depth and structural cues where RGB alone struggles — e.g., poor lighting, heavy occlusion, or object truncation.

Impact of LiDAR Beam Density: KITTI vs nuScenes

- **KITTI** uses a high-resolution 64-beam Velodyne HDL-64E → denser point clouds → richer 2D/BEV projections.
- **nuScenes** uses a 32-beam Velodyne VLP-32C → sparser, less informative LiDAR representation.
- Sparse LiDAR in nuScenes limits the effectiveness of fusion, especially for small/distant objects — visible in reduced performance across difficult categories.

Challenges in 3D Object Detection

- **High false positive rate** due to overconfident bounding boxes with poor localization.
- **Non-Max Suppression (NMS)** struggles when predictions have high overlap or confidence but incorrect positions.
- **Loss functions (like CloU)** are still evolving for 3D and orientation-aware scenarios — tuning them remains a work-in-progress.

Dataset Characteristics Affect Generalization

- **KITTI**: Small dataset (~7.5k samples), well-balanced with high-quality annotations.
- **nuScenes**: Large-scale (~6.7k test samples), more classes (23 vs KITTI's 8), leading to **class imbalance** and more complex training dynamics.
- Fusion strategies must adapt to these dataset-specific characteristics for robust results.

Cross-Dataset Adaptability is a Major Challenge

- YOLOv3 and fusion models trained on one dataset (e.g., KITTI) fail when tested on another (e.g., nuScenes), especially for LiDAR-based features.
- Root cause: **sensor disparity** — 64-beam vs 32-beam → mismatched point cloud density, spatial coverage, and vertical resolution.
- Without adaptive normalization or sensor-aware domain alignment, **LiDAR fusion does not generalize across datasets**.

Limitations in Data Sampling & Nighttime Performance

- **nuScenes split is blob-based**: a scene belongs entirely to either train or test → no intra-scene generalization.
- As a result, **nighttime data was missing during training**, leading to poor performance in night splits — especially visible in Cyclist and Pedestrian detection.
- Future work should explore **per-blob sampling** to improve temporal and condition diversity across splits.



Thank You